Feature engineering

Feature engineering is the process of using domain knowledge to select, modify, or create new input features from raw data to improve the performance of machine learning models.

Key Steps in Feature Engineering

1. **Understanding the Data**
   - Know your dataset: distributions, types (numerical, categorical, text), missing values, etc.
   - Understand domain relevance of each feature.
2. **Feature Selection**
   - Remove irrelevant or redundant features.
   - Use techniques like:
     - Correlation analysis
     - Mutual information
3. **Feature Transformation**
   - **Scaling**: StandardScaler, MinMaxScaler
   - **Normalization**: Especially for distance-based models
   - **Log transforms**: For skewed data
4. **Feature Encoding**
   - Convert categorical variables:
     - One-Hot Encoding
     - Label Encoding
5. **Feature Creation**
   - **Date features**: Extract day, month, hour, weekend, etc.
   - **Text features**: Word counts, TF-IDF, sentiment

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Sreeja Homes, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 97 [illegible] 1912 5

- **Combinations**: Ratios, interactions between features
- **Aggregation**: For time series or grouped data (e.g., mean sales per store)

6. **Handling Missing Data**
   - Imputation (mean, median, mode, KNN, etc.)
   - Flagging missingness as a separate feature
7. **Dimensionality Reduction**
   - PCA, t-SNE, UMAP (mostly for visualization or complex data)
   - Useful when too many features are generated

Feature Creation (*few mentioned here)

Feature creation is a key step in the machine learning pipeline where raw data is transformed into a set of measurable and informative inputs (features) that can be used to train a model. Extracting the features means deriving from an existing features for machine learning.

Feature Creation using domain knowledge involves leveraging expert knowledge of a particular field to create meaningful features that can improve the performance of machine learning models. These features are designed to capture the underlying patterns or important aspects of the data that might not be immediately obvious from raw features alone.

1. Financial Data: Price-to-Earnings (P/E) Ratio

In financial data, a commonly used feature is the **Price-to-Earnings (P/E) ratio**, which is calculated by dividing the stock price by the earnings per share (EPS). This ratio is often used by investors to assess whether a stock is overvalued or undervalued.

```
import pandas as pd
# Sample financial data
data = {'Stock_Price': [100, 120, 140, 160, 180],\
        'Earnings_Per_Share': [5, 6, 7, 8, 9]}

df = pd.DataFrame(data)

# Calculate the Price-to-Earnings (P/E) ratio
df['PE_Ratio'] = df['Stock_Price'] /
df['Earnings_Per_Share']
print(df)
```

Output:

	Stock_Price	Earnings_Per_Share	PE_Ratio
0	100	5	20.0
1	120	6	20.0
2	140	7	20.0
3	160	8	20.0
4	180	9	20.0

In this example, the **PE_Ratio** is an important financial feature that can help assess the stock's valuation.

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

2. Healthcare Data: Body Mass Index (BMI)

In healthcare, the **Body Mass Index (BMI)** is a useful feature for classifying individuals as underweight, normal weight, overweight, or obese. The formula for BMI is:

BMI= Weight(kg)/Height(m)**2

```
import pandas as pd
# Sample healthcare data
data = {'Weight': [60, 70, 80, 90, 100],   # in kg
        'Height': [1.60, 1.70, 1.75, 1.80, 1.85]}# in meters

df = pd.DataFrame(data)
# Calculate Body Mass Index (BMI)
df['BMI'] = df['Weight'] / (df['Height'] ** 2)

# Add BMI category based on value ranges
def bmi_category(bmi):
    if bmi < 18.5:
        return 'Underweight'
    elif 18.5 <= bmi < 24.9:
        return 'Normal weight'
    elif 25 <= bmi < 29.9:
        return 'Overweight'
    else:
        return 'Obese'
```

smetechnologies
GUIDING THE FUTURE WITH A
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
df['BMI_Category'] = df['BMI'].apply(bmi_category)
print(df)
```

Output:

	Weight	Height	BMI	BMI_Category
0	60	1.60	23.437500	Normal weight
1	70	1.70	24.221453	Normal weight
2	80	1.75	26.122449	Overweight
3	90	1.80	27.777778	Overweight
4	100	1.85	29.221391	Overweight

In this example, **BMI** is a feature used to classify individuals' body weight, which can be useful for health predictions.

3. Retail Data: Day of the Week (for Sales Prediction)

In retail data, sales often depend on the day of the week, as consumer buying behavior varies on weekdays versus weekends. A simple feature is to extract the **day of the week** from the date of a sale.

```
import pandas as pd
# Sample datetime column
data = {'timestamp': ['2023-01-01 12:00:00',\
                      '2023-02-01 14:30:00', '2023-03-01 16:45:00']}
df = pd.DataFrame(data)
df['timestamp'] = pd.to_datetime(df['timestamp'])

# Extract datetime features
```

smetechnologies
GEARING THE FUTURE WITH AI
Floor, Express House, Kochi, Ernakulam, Kerala 682017
No: +91 9778191725

```
df['year'] = df['timestamp'].dt.year
df['month'] = df['timestamp'].dt.month
df['day'] = df['timestamp'].dt.day
df['hour'] = df['timestamp'].dt.hour
df['minute'] = df['timestamp'].dt.minute
df['weekday'] = df['timestamp'].dt.weekday  #0=Monday, 6 =
#Sunday
print(df)
```

Output:

```
timestamp  year  month  day  hour  minute  weekday

0 2023-01-01 12:00:00  2023      1    1    12      0        6

1 2023-02-01 14:30:00  2023      2    1    14      30       2

2 2023-03-01 16:45:00  2023      3    1    16      45       2
```

4. E-commerce Data: Price Range

In e-commerce, **Price Range** can be an important feature to help predict whether a product will sell based on its price category (cheap, medium, expensive). The feature can be extracted by categorizing the price values into different ranges.

```
import pandas as pd
```

smectechnologies
GUIDING THE FUTURE WITH
1st Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph No: +91 9778191215

```
# Sample e-commerce data with product prices
data = {'Product': ['A', 'B', 'C', 'D', 'E'],\
        'Price': [15, 100, 250, 50, 300]}

df = pd.DataFrame(data)
# Define price categories (cheap, medium, expensive)
def price_range(price):
    if price < 50:
        return 'Cheap'
    elif 50 <= price < 200:
        return 'Medium'
    else:
        return 'Expensive'

df['Price_Range'] = df['Price'].apply(price_range)

print(df)
```

Output:

	Product	Price	Price_Range
0	A	15	Cheap
1	B	100	Medium
2	C	250	Expensive
3	D	50	Medium
4	E	300	Expensive

smectechnologies
GUIDING THE FUTURE WITH AI
Kaloor, Kochi, Ernakulam, Kerala 682017
+91 9778191215

In this case, the **Price_Range** feature categorizes the products into price brackets, which can be important for predicting customer behavior or product sales.

5. Weather Data: Temperature Ranges

In weather data, temperature ranges can be used as features to determine specific weather conditions (like hot, warm, cold). This is useful for models predicting seasonal behaviors or events that depend on temperature, such as power consumption or outdoor activity planning.

```
import pandas as pd
# Sample weather data with temperatures (in Celsius)
data = {'Date': ['2025-04-01', '2025-04-02',\
                 '2025-04-03', '2025-04-04'],\
        'Temperature': [25, 15, 30, 10]}

df = pd.DataFrame(data)

# Define temperature categories
def temperature_range(temp):
    if temp < 15:
        return 'Cold'
    elif 15 <= temp < 25:
        return 'Warm'
    else:
        return 'Hot'
df['Temperature_Range'] = \
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 97 8191215

```
            df['Temperature'].apply(temperature_range)
print(df)
```

Output:

```
         Date  Temperature Temperature_Range
0  2025-04-01           25              Hot
1  2025-04-02           15             Warm
2  2025-04-03           30              Hot
3  2025-04-04           10             Cold
```

Here, **Temperature_Range** is a domain-specific feature that categorizes temperature values, which could help predict outdoor activities or energy usage.

Feature extraction based on Mathematical and Statistical Transformations (*few mentioned)

Mathematical transformations can be used to create new features or modify existing ones to highlight specific patterns or relationships.

1. Log Transformations

log transformation to the data to reduce skewness, especially when the data has a **positive skew** (where the tail of the distribution is towards higher values).

A log transformation can help compress large values and spread out smaller values. This is useful in cases where your data has a significant positive skew.

```
import numpy as np
import pandas as pd

# Sample data with a positive skew
data = {'Value': [1, 4, 9, 16, 25]}

df = pd.DataFrame(data)

# Apply log transformation (natural log)
df['Log_Value'] = np.log(df['Value'])

# If the data contains zeros or negative values, add a
constant (e.g., 1) before applying log.
# This avoids issues with undefined values in log(0) or
negative numbers.
df['Log_Value_safe'] = np.log(df['Value'] + 1)  # Adding 1
#ensures no log(0) issue.

print(df)
```

Output:

Value Log_Value Log_Value_safe

smetechnologies
GUIDING THE FUTURE WITH AI
II floor, Cypress House, Palloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191415

```
0      1    0.000000        0.693147

1      4    1.386294        1.609438

2      9    2.197225        2.302585

3     16    2.772589        2.833213

4     25    3.218876        3.258097
```

2. Custom Transformations

Custom transformations are useful when you have domain-specific knowledge or a specific way to preprocess data.

Binning

Binning is a technique where you group continuous data into discrete intervals or "bins." This is useful for categorical data when dealing with continuous variables, such as age ranges or income groups.

```
import pandas as pd

# Sample continuous data
data = {'Age': [12, 35, 45, 23, 67, 89]}
df = pd.DataFrame(data)
# Define bins
bins = [0, 18, 35, 55, 100]
labels = ['0-18', '19-35', '36-55', '56+']
# Create a binned feature
```

smetechnologies
GROWING THE FUTURE WITH AI
II Floor, Express House, Kaipor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
df['Age_Group'] = pd.cut(df['Age'], bins=bins,  \
                                    labels=labels)
print(df)
```

Output:

```
   Age Age_Group
0   12      0-18
1   35     19-35
2   45     36-55
3   23     19-35
4   67       56+
5   89       56+
```

In this example, **Age_Group** is a binned version of the **Age** column, categorizing people into different age ranges.

3. Categorical Encoding

For categorical variables, you can convert them into numerical form using methods like **One-Hot Encoding** or **Label Encoding**, which allows machine learning algorithms to process categorical data.

One-Hot Encoding Example

```
import pandas as pd

# Sample data with a categorical feature
data = {'Category': ['A', 'B', 'A', 'C', 'B']}

df = pd.DataFrame(data)
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9746819215

```
# Apply One-Hot Encoding
df = pd.get_dummies(df, columns=['Category'])

print(df)
```

Output:

	Category_A	Category_B	Category_C
0	1	0	0
1	0	1	0
2	1	0	0
3	0	0	1
4	0	1	0

Here, the **Category** feature is converted into binary features for each category.

4. Label Encoding

Pandas also provides a method, which encodes categorical variables into integer labels:

```
import pandas as pd

# Sample DataFrame
data = {'Category': ['cat', 'dog', 'dog', 'cat', \
                     'rabbit', 'rabbit']}
df = pd.DataFrame(data)

# Encode the 'Category' column
df['Category_Encoded'] = pd.factorize(df['Category'])[0]
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
print(df)
```

Output:

```
  Category  Category_Encoded
0      cat                 0
1      dog                 1
2      dog                 1
3      cat                 0
4   rabbit                 2
5   rabbit                 2
```

Feature Engineering: Scaling, Normalization, and Standardization

Feature Scaling is a critical step in building accurate and effective machine learning models. Feature scaling is a preprocessing technique that transforms feature values to a similar scale, ensuring all features contribute equally to the model. It's essential for datasets with features of varying ranges, units, or magnitudes. Common techniques include standardization, normalization, and min-max scaling. This process improves model performance, convergence, and prevents bias from features with larger values

Standardization

Each feature is transformed to have a mean of 0 and a standard deviation of 1. This is achieved by subtracting the mean value and dividing by the standard deviation of the feature.

The formula for standardization is:

$$Z = (x - \mu) / \sigma$$

- Z is the standardized score (also called a z-score)
- x is the original value you want to standardize
- μ (mu) is the mean of the data set
- σ (sigma) is the standard deviation of the data set
  - Consider a dataset with age data for three individuals:
  - - Person A: 45 years
    - Person B: 44 years
    - Person C: 40 years
  - To standardize the age values using the formula:
    - Mean (μ) = $\frac{45+44+40}{3} = 43$
    - Standard deviation (σ) = $\sqrt{\frac{(45-43)^2+(44-43)^2+(40-43)^2}{3}}$

      $\sigma = \sqrt{\frac{4+1+9}{3}} = \sqrt{\frac{14}{3}} \approx 1.53$
  - 2. Apply the standardization formula to each age value:

smetechnologies
GUIDING THE FUTURE WITH
II floor, Express House, Kaloor, Kochi, Ernakulam, Kerala ©2017
Ph. No: +91 97781911 15

- For Person A:
  $$x' = \frac{45-43}{1.53} \approx \frac{2}{1.53} \approx 1.31$$
- For Person B:
  $$x' = \frac{44-43}{1.53} \approx \frac{1}{1.53} \approx 0.65$$
- For Person C:
  $$x' = \frac{40-43}{1.53} \approx \frac{-3}{1.53} \approx -1.96$$

- After standardization:
  - Person A's standardized age ≈ 1.31
  - Person B's standardized age ≈ 0.65
  - Person C's standardized age ≈ -1.96
- Standardization transforms data to have properties that are more suitable for many machine learning algorithms. By centering the data around 0 and scaling it to have a standard deviation of 1, standardization ensures that all features contribute equally to model training and evaluation. This preprocessing step is particularly useful when features have varying scales and distributions, enhancing the stability and performance of machine learning models. Skewed data and outliers can negatively impact the performance of machine learning models. Scaling the features can help in handling such cases. By transforming the data to a standardized range, it reduces the impact of extreme values and makes the model more robust

```
# import necessary libraries
import pandas as pd
# Create a sample DataFrame
data = {       \
```

```
        'Feature1': [1, 3, 5, 7, 9],\
        'Feature2': [2, 4, 6, 8, 10]\
    }
    df = pd.DataFrame(data)

    # Manually standardize just 'Feature1'
    df['Feature1_standardized'] = (df['Feature1'] - \
    df['Feature1'].mean()) / df['Feature1'].std()
    # Display the original and standardized DataFrame
    print("Original DataFrame:\n", df[['Feature1', \
                                       'Feature2']])
    print("\nDataFrame with Standardized 'Feature1':\n",
    df[['Feature1', 'Feature1_standardized', 'Feature2']])
```

Output:

Original DataFrame:

	Feature1	*Feature2*
0	*1*	*2*
1	*3*	*4*
2	*5*	*6*
3	*7*	*8*

smectechnologies
GUIDING THE FUTURE WITH
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala
Ph. No: +91 9778191215

```
4          9         10

DataFrame with Standardized 'Feature1':

     Feature1  Feature1_standardized  Feature2

0          1              -1.264911         2

1          3              -0.632456         4

2          5               0.000000         6

3          7               0.632456         8

4                    9                1.264911          10
```

Min Max scaling

This method scales each feature so that all values are within the range of 0 and 1. It achieves this by subtracting the minimum value of the feature and dividing by the range (difference between maximum and minimum values).

Fundamental technique in machine learning that transforms numerical features to a standard scale, typically between 0 and 1.

This process ensures that all features contribute equally to the analysis, preventing those with larger numerical ranges from dominating the model's

smetechnologies GUIDING THE FUTURE WITH AI

outcomes. This method is sensitive to outliers as it uses the minimum and maximum values of the features

$$(x - min(X)) / (max(X) - min(X))$$

- x is the value you want to normalize
- min(X) is the minimum value in your data set
- max(X) is the maximum value in your data set

```
import pandas as pd
# Create a sample DataFrame
data = {
    'Feature1': [1, 3, 5, 7, 9],
    'Feature2': [2, 4, 6, 8, 10]
}
df = pd.DataFrame(data)

# Manually normalize 'Feature1' using Min-Max
normalization
min_value = df['Feature1'].min()
max_value = df['Feature1'].max()

df['Feature1_normalized'] = (df['Feature1'] - min_value) /
(max_value - min_value)
```

smctechnologies
GLOBAL THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
# Display the original and normalized DataFrame
print("Original DataFrame:\n", df[['Feature1', \
                                   'Feature2']])
print("\nDataFrame with Normalized 'Feature1':\n",
df[['Feature1', 'Feature1_normalized', 'Feature2']])
```

Output:

Original DataFrame:

	Feature1	Feature2
0	1	2
1	3	4
2	5	6
3	7	8
4	9	10

DataFrame with Normalized 'Feature1':

	Feature1	Feature1_normalized	Feature2
0	1	0.00	2
1	3	0.25	4

smectechnologies
GUIDING THE FUTURE WITH AI
1st Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

2	5	0.50	6
3	7	0.75	8
4	9	1.00	10

Mean Normalization

Mean Normalization centers the features around the mean, ensuring that the transformed data has a mean of zero. It scales the features to a range between -1 and 1. This is similar to MixMax scaling but difference mean is subtracted from the value than the minimum value

Use Mean Normalization when you want to center the data around the mean and scale it between -1 and 1. It is helpful when the data has a known range but varies around the mean.

$$(x - mean(X)) / (max(X) - min(X))$$

- x is the value you want to normalize
- min(X) is the minimum value in your data set
- max(X) is the maximum value in your data set

```
import pandas as pd

# Sample data
```

smctechnologies
GUIDING THE FUTURE WITH AI
Kaloor, Express Hotels, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph No: +91 7781915

```
data = {
    'Feature1': [1, 3, 5, 7, 9],
    'Feature2': [2, 4, 6, 8, 10]
}
# Create a DataFrame
df = pd.DataFrame(data)
# Calculate the mean, min, and max of 'Feature1'
mean_feature1 = df['Feature1'].mean()
min_feature1 = df['Feature1'].min()
max_feature1 = df['Feature1'].max()

# Apply mean normalization to 'Feature1'
df['Feature1_normalized'] =  \
    (df['Feature1']-mean_feature1)/(max_feature1- \
     min_feature1)
# Display the result
print("Original DataFrame:\n", df[['Feature1', 'Feature2']])
print("\nDataFrame with Mean Normalized 'Feature1':\n",
df[['Feature1', 'Feature1_normalized', 'Feature2']])
```

Output:

Original DataFrame:

	Feature1	*Feature2*
0	*1*	*2*
1	*3*	*4*

smetechnologies
GUIDING TO FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7781121212

```
2          5          6

3          7          8

4          9         10

DataFrame with Mean Normalized 'Feature1':

    Feature1  Feature1_normalized  Feature2

0          1                -0.50         2

1          3                -0.25         4

2          5                 0.00         6

3          7                 0.25         8

4          9                 0.50        10
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215